

Joint versus Separate Optimisation: 100-Replication Simulation Review

gamlss.longitudinal JSS replication notes

Compiled from outputs completed 2026-06-14

1 Purpose

This note summarises the 100-replication simulation comparing the two random-search optimisation strategies used in the JSS replication workflow: `rs_joint` and `rs_separate`. The goal was to test whether joint optimisation gives a practically meaningful improvement over separate optimisation once the iteration caps are increased enough to give the joint method a fairer convergence opportunity.

2 Run Provenance

The completed checkpointed run wrote the final combined outputs on **2026-06-14 22:49:15 +1000**. The checkpointing design wrote one CSV per replication before combining results, so machine restarts no longer discard all in-memory progress.

Table 1: Completed run artifacts.

Item	Value
Replications	100
Rows per replication	36
Total result rows	3600
<code>rs_joint</code> fits	1800
<code>rs_separate</code> fits	1800
Checkpoint directory	results/jss-replication/expanded/checkpoints/03-joint-vs-separate-optimiza
Final result CSV	results/jss-replication/expanded/data/03-joint-vs-separate-optimization-re
Summary table	results/jss-replication/expanded/tables/03-joint-vs-separate-optimization-
Metric-win table	results/jss-replication/expanded/tables/03-joint-vs-separate-optimization-

3 Simulation Design

The formal run used the expanded profile for module 03, `03-joint-vs-separate-optimization`. The simulation grid was built with `run_coverage.simulations()` and then fit with only the two longitudinal random-search methods, `rs_separate` and `rs_joint`. Table 2 lists the settings used in the completed run.

Table 2: Simulation settings used in the 100-replication run.

Setting	Value and role
profile	expanded; writes under <code>results/jss-replication/expanded</code> .
families	NO, NBI. NO is the normal response family; NBI is the negative-binomial type I count family.
copulas	N, C, G. These are Gaussian, Clayton, and Gumbel copulas.
designs	intercept, covariate, time_dependence. Details are given below.
n	48 subjects per simulated data set.
times	Four repeated measurements, $\{1, 2, 3, 4\}$, per subject before missingness.
dependence	strong; used for constant-dependence designs.
missingness	none; no responses were removed after simulation.
methods	rs_separate, rs_joint.
start_mode	default; no truth-adjacent warm start was requested.
max_outer_iter	16.
max_inner_iter	8.
max_elapsed_sec	120 seconds per fit attempt.
base_seed	20260528.

3.1 Response Families

The response distribution was generated from `gamlss.dist` families. For the two families used here, the coverage simulator applies conservative default parameters:

- NO: $\mu = 0$, $\sigma = 1$.
- NBI: $\mu = 4$, $\sigma = 0.5$.

These base values are used directly in the `intercept` and `time_dependence` designs. In the `covariate` design, the μ parameter is modified by a subject-level covariate as described below.

3.2 Copula Families and Dependence Strength

The simulator uses a first-order longitudinal copula model between adjacent time points. The included copula codes were:

- N: Gaussian copula, parameterised by a correlation-like θ with Fisher- z link in the fitted model.
- C: Clayton copula, parameterised internally through Kendall's τ in the simulator and a positive θ link in the fitted model.
- G: Gumbel copula, also generated from a Kendall's τ path and fit through the Gumbel parameter link.

For constant-dependence designs, the `strong` dependence setting maps to $\rho = 0.65$ for the Gaussian copula and $\tau = 0.55$ for the Clayton and Gumbel copulas. For the `time_dependence` design, the constant `strong` setting is replaced by a deterministic time-varying dependence path:

- N: $\theta(t) = 0.1 + 0.45 s(t)$, where $s(t)$ rescales the left endpoint of each adjacent time pair to $[0, 1]$.

- **C** and **G**: $\tau(t) = 0.1 + 0.3 s(t)$.

With four time points, the adjacent pair left endpoints are 1, 2, 3, so the time-varying path is evaluated over three adjacent-pair dependence levels.

3.3 Designs

The three design settings control which part of the data-generating model is non-constant and which formulas are used when fitting each method.

intercept. All marginal parameters and the copula parameter are constant. The fitted formulas are intercept-only for the margins and dependence:

$$\mu \sim 1, \quad \sigma \sim 1, \quad \nu \sim 1, \quad \tau \sim 1, \quad \theta \sim 1, \quad \zeta \sim 1.$$

covariate. A subject-level covariate x is generated by rescaling the subject index to $[0, 1]$ and then centering it. The data-generating μ parameter varies with this covariate. For positive baseline means, the simulator uses

$$\mu(x) = \mu_0 \exp(0.15x),$$

while for non-positive baseline means it uses

$$\mu(x) = \mu_0 + 0.15x.$$

Thus, for **NO**, $\mu(x) = 0.15x$; for **NBI**, $\mu(x) = 4 \exp(0.15x)$. The fitted formulas use x only in the mean submodel:

$$\mu \sim x, \quad \sigma \sim 1, \quad \nu \sim 1, \quad \tau \sim 1, \quad \theta \sim 1, \quad \zeta \sim 1.$$

time_dependence. The marginal response model remains intercept-only, but the adjacent-time copula dependence changes across time. The fitted dependence formula is

$$\theta \sim \text{time},$$

with the other marginal parameters held at intercept-only formulas. This design is the one that directly tests recovery of time-varying dependence; it is also why **benchmark_theta_time_abs_error** is finite only for the time-dependence scenarios.

3.4 Replication and Seeding

For replication r , the module called **run_coverage_simulations()** with seed

$$20260528 + 3000 + 100r.$$

Inside the coverage grid runner, the individual case seed is offset again by the case index i , giving effective case-level seeds

$$20260528 + 3000 + 100r + i.$$

This made each family-copula-design-method cell reproducible while avoiding identical random streams across cells.

3.5 Optimisation Settings

Both methods were run with the same outer, inner, and elapsed-time caps. The `rs_separate` method optimises marginal and dependence components in a separate/backfitting-style sequence, while `rs_joint` evaluates the joint marginal-plus-copula objective in the random-search loop. The increased `max_outer_iter` = 16 and `max_elapsed_sec` = 120 settings were chosen after the 10-replication calibration showed that the lower-cap joint run had avoidable non-convergence.

Each replication therefore contained

$$2 \text{ families} \times 3 \text{ copulas} \times 3 \text{ designs} \times 2 \text{ methods} = 36$$

fits. Across 100 replications, this gives 1800 paired comparison cells and 3600 fitted-model rows.

4 Candidate Set

The module first mined retained exploratory summaries and selected seven candidate cases for review. These candidates determined the copula set used by the formal grid. Five were classified as `joint_win`, one as `tie_control`, and one as `cautionary`.

Table 3: Selected candidate sources.

Candidate	Role	Copula	Source
jvs-01	joint_win	N	top_cases_loglik_logscore_all_rmse_improved
jvs-02	joint_win	N	stress_balanced_candidates
jvs-03	joint_win	C	top_cases_loglik_logscore_all_rmse_improved
jvs-04	joint_win	N	stress_balanced_candidates
jvs-05	joint_win	N	stress_balanced_candidates
jvs-06	tie_control	G	top_cases_loglik_logscore_all_rmse_improved
jvs-07	cautionary	C	joint_vs_separate_delta_summary

5 Overall Completion, Convergence, and Runtime

Both methods completed most fits, but the separate optimiser retained a clear convergence advantage. The joint optimiser’s convergence improved relative to the earlier low-cap pilot, but did not reach parity with the separate method.

Table 4: Overall method-level completion and runtime.

Method	Fits	Success rate	Convergence rate	Median sec.	Mean sec.
<code>rs_joint</code>	1800	0.966	0.861	5.10	15.68
<code>rs_separate</code>	1800	0.983	0.982	1.71	9.35

5.1 Additional Fit Outcomes

The result table also records diagnostic fit outcomes beyond success, convergence, and runtime. These include convergence-timeout classifications, warnings, marginal fallback use, likelihood com-

Table 5: Completion and runtime by response family.

Method	Family	Success rate	Convergence rate	Median sec.	Mean sec.
<code>rs_joint</code>	NBI	0.932	0.863	13.14	28.17
<code>rs_separate</code>	NBI	0.967	0.967	5.67	17.68
<code>rs_joint</code>	NO	1.000	0.858	2.31	3.19
<code>rs_separate</code>	NO	1.000	0.998	0.74	1.02

ponents, parameter recovery errors, and fitted dependence summaries. Table 6 summarises the most informative populated fields.

Table 6: Additional method-level fit outcomes.

Outcome	<code>rs_joint</code>	<code>rs_separate</code>
Convergence-timeout rate	0.034	0.017
Warning rate	0.188	0.083
Marginal fallback rate	0.000	0.000
Median joint log-likelihood	-269.50	-273.19
Median marginal log-likelihood	-297.26	-299.36
Median copula log-likelihood	41.63	40.29
Median max. absolute parameter error	0.333	0.336
Median absolute copula- τ error	0.055	0.057
90th percentile elapsed sec.	42.14	23.44

All recorded failures were classified as convergence timeouts; there were no marginal fallback fits in either method, and all fits used the default attempt path. The unpaired method-level medians suggest slightly higher likelihoods and slightly smaller parameter errors for `rs_joint`, but the paired finite deltas were close to neutral: mean paired joint-log-likelihood delta was -0.32 , median paired joint-log-likelihood delta was -0.35 , and joint won 49.1% of finite paired likelihood comparisons. For maximum absolute parameter error, joint had a small paired advantage, with mean separate-minus-joint error 0.0031, median 0.0029, and a 51.0% joint win rate. For absolute copula- τ error, the sign reversed slightly: mean separate-minus-joint error was -0.0021 , median -0.0012 , and the joint win rate was 48.9%.

6 Metric-Win Results

The metric-win table compares methods within paired simulation cells. For error and scoring metrics, lower raw values are better; for `benchmark_pit_ks_p_value`, larger values are better. Runtime is also treated as a metric where lower is better. Some metrics have fewer finite comparisons because failures or non-finite benchmark quantities remove paired cells from that metric.

7 Scenario-Level Deltas

Table 8 reports scenario-level averages for the paired comparison. The delta columns use the sign convention:

Table 7: Selected metric win and win-or-tie rates.

Metric	Method	Finite n	Wins	Win rate	Win/tie rate
neg_log_score	rs_joint	1739	871	0.492	0.785
neg_log_score	rs_separate	1770	901	0.508	0.796
mean_rmse	rs_joint	1739	879	0.496	0.517
mean_rmse	rs_separate	1770	893	0.504	0.517
q90_mae	rs_joint	1739	1032	0.582	0.597
q90_mae	rs_separate	1770	1021	0.576	0.589
theta_time_abs_error	rs_joint	600	309	0.515	0.530
theta_time_abs_error	rs_separate	600	291	0.485	0.495
lower_tail_error_05	rs_joint	1739	1309	0.739	0.739
lower_tail_error_05	rs_separate	1770	1387	0.783	0.783
upper_tail_error_05	rs_joint	1739	939	0.530	0.532
upper_tail_error_05	rs_separate	1770	1027	0.580	0.580
pit_mean_abs_error	rs_joint	1739	792	0.447	0.507
pit_mean_abs_error	rs_separate	1770	980	0.553	0.601
interval_coverage_95	rs_joint	1739	960	0.542	0.542
interval_coverage_95	rs_separate	1770	1089	0.615	0.615
elapsed_sec	rs_joint	1800	117	0.065	0.081
elapsed_sec	rs_separate	1800	1683	0.935	0.943

$$\Delta_{\text{error}} = \text{separate error} - \text{joint error},$$

so positive values favour the joint method for error and score metrics. For joint log-likelihood,

$$\Delta_{\log L} = \log L_{\text{joint}} - \log L_{\text{separate}},$$

so positive values also favour the joint method. For elapsed time,

$$\Delta_{\text{time}} = \text{separate sec.} - \text{joint sec.},$$

so negative values indicate that the joint method was slower.

Table 8: Scenario-level paired deltas, ordered by core win rate.

Family	Copula	Design	Joint conv.	Sep. conv.	Core win	Δ NLS	Δ mean RMSE	Δ q90	$\Delta\theta(t)$	Δ sec.
NO	G	intercept	0.74	1.00	0.533	-0.011	0.018	0.021	—	-3.219
NO	N	intercept	1.00	1.00	0.525	0.013	0.019	0.011	—	-1.476
NO	C	time_dependence	1.00	0.99	0.508	0.000	0.001	-0.001	0.040	-1.396
NO	G	time_dependence	1.00	0.99	0.504	-0.007	0.012	0.003	-0.019	-1.307
NO	C	intercept	0.58	1.00	0.498	0.010	-0.013	0.016	—	-2.973
NO	N	covariate	1.00	1.00	0.495	-0.017	0.010	0.007	—	-1.452
NO	G	covariate	0.77	1.00	0.478	0.004	0.012	0.006	—	-3.263
NO	N	time_dependence	1.00	1.00	0.464	0.011	-0.003	-0.007	-0.006	-0.674
NO	C	covariate	0.63	1.00	0.455	-0.012	-0.014	-0.003	—	-3.751
NBI	G	time_dependence	0.98	0.98	0.400	0.034	0.042	0.020	0.034	-2.776
NBI	N	covariate	0.43	0.73	0.378	-0.017	0.024	0.062	—	-23.441
NBI	N	time_dependence	1.00	1.00	0.362	0.005	0.027	-0.060	0.013	-18.264
NBI	C	intercept	0.89	1.00	0.360	-0.004	0.044	0.180	—	-5.776
NBI	G	intercept	0.85	1.00	0.355	-0.004	0.095	0.180	—	-6.504
NBI	C	time_dependence	0.99	0.99	0.352	-0.007	0.014	0.160	-0.058	-2.882
NBI	N	intercept	1.00	1.00	0.348	0.003	0.025	0.090	—	-17.566
NBI	G	covariate	0.76	1.00	0.313	-0.010	-0.140	-0.153	—	-9.353
NBI	C	covariate	0.87	1.00	0.295	-0.012	-0.089	-0.118	—	-7.847

8 Interpretation

The 100-replication run gives a more conservative picture than the earlier 10-replication calibration. The joint method is competitive on selected prediction and dependence-recovery metrics, but it does not dominate the separate method overall.

Where joint optimisation helped most. The clearest joint-favouring scenarios were in the Gaussian/normal family. The `NO + G + intercept` and `NO + N + intercept` scenarios had the highest core win rates. The `NO + C + time_dependence` case is still a useful example because both methods converged reliably and the joint method showed a positive mean gain on the time-varying dependence error. Across all finite paired cells, the joint method had a small edge on `benchmark_q90_mae` and `benchmark_theta_time_abs_error`.

Where separate optimisation remained stronger. The separate method remained substantially faster. It won or tied on runtime in 94.3% of paired comparisons, while the joint method won or tied in only 8.1%. Separate optimisation also retained better global convergence: 98.2% versus 86.1%. Calibration and tail metrics generally leaned separate, especially `benchmark_pit_mean_abs_error`, `benchmark_tail_error_lower_05`, and `benchmark_tail_error_upper_05`.

Family-level pattern. The NBI scenarios were materially harder. Joint optimisation had lower success and convergence rates for NBI, and some `NBI + N` cells were especially slow. Even where the joint method improved some predictive metrics in NBI, it often did so with much larger runtime costs and weaker convergence.

Bottom line. The full run supports a nuanced conclusion: increasing the iteration cap made joint optimisation viable enough for review, but it did not make joint optimisation the uniformly preferred method. Joint optimisation is best framed as a targeted option that can improve upper-quantile prediction or time-varying dependence recovery in some Gaussian scenarios, while separate optimisation remains the more stable and computationally efficient default.

9 Figures

The completed run also produced two diagnostic figures:

- `results/jss-replication/expanded/figures/03-joint-vs-separate-optimization-deltas.png`
- `results/jss-replication/expanded/figures/03-joint-vs-separate-optimization-metric-dashboar`

These are not embedded here to keep the LaTeX source portable across working directories, but they should be reviewed alongside this note.